Can LLMs Classify CVEs? Investigating LLMs Capabilities in Computing CVSS Vectors

Francesco Marchiori
Department of Mathematics
University of Padova
Padua, Italy
francesco.marchiori.4@phd.unipd.it

Denis Donadel
Department of Computer Science
University of Verona
Verona, Italy
denis.donadel@univr.it

Mauro Conti
Department of Mathematics
University of Padova
Padua, Italy
mauro.conti@unipd.it

2025 IEEE Symposium on Computers and Communications (ISCC) | 979-8-3315-2420-3/25/$31.00 ©2025 IEEE | DOI: 10.1109/ISCC65549.2025.11326354

***Abstract*—Common Vulnerability and Exposure (CVE) records are fundamental to cybersecurity, offering unique identifiers for publicly known software and system vulnerabilities. Each CVE is typically assigned a Common Vulnerability Scoring System (CVSS) score to support risk prioritization and remediation. However, score inconsistencies often arise due to subjective interpretations of certain metrics. As the number of new CVEs continues to grow rapidly, automation is increasingly necessary to ensure timely and consistent scoring. While prior studies have explored automated methods, the application of Large Language Models (LLMs), despite their recent popularity, remains relatively underexplored.**

In this work, we evaluate the effectiveness of LLMs in generating CVSS scores for newly reported vulnerabilities. We investigate various prompt engineering strategies to enhance their accuracy and compare LLM-generated scores against those from embedding-based models, which use vector representations classified via supervised learning. Our results show that while LLMs demonstrate potential in automating CVSS evaluation, embedding-based methods outperform them in scoring more subjective components, particularly confidentiality, integrity, and availability impacts. These findings underscore the complexity of CVSS scoring and suggest that combining LLMs with embedding-based methods could yield more reliable results across all scoring components.

***Index Terms*—Large Language Models, Embeddings, Cyber Threat Intelligence, CVE, CVSS**

I. Introduction

In cybersecurity, managing and addressing vulnerabilities is crucial to maintaining systems and data integrity. The Common Vulnerabilities and Exposure (CVE) system is a standardized method for identifying and cataloging publicly known security vulnerabilities, providing a common reference point for organizations to communicate risks. Alongside CVE, the Common Vulnerability Scoring System (CVSS) plays a key role in evaluating the severity of these vulnerabilities, enabling organizations to prioritize remediation efforts. It is generated in the form of a vector of elements representing different features of the CVE that can be summarized in a single score (CVSS score), making it a valuable index for prioritization. Together, CVE and CVSS offer essential tools for identifying, understanding, and addressing vulnerabilities consistently and structured, helping organizations mitigate potential security threats more effectively.

Generating the CVSS vector (and consequently CVSS scores) from CVEs is a demanding task for analysts [1]. Not only the number of CVEs has increased by 38% from 2023 to 2024 [2], but the final score can also differ based on the subjective nature of some of the CVSS vector, such as the level of confidentiality, integrity, and availability risk. With more than 135 new CVEs published each day [3], automation is becoming increasingly essential to streamline the process of scoring vulnerabilities, ensuring faster response times, reducing human error, and enhancing the overall efficiency of cybersecurity operations.

Artificial Intelligence (AI) may help in this task by providing suggestions to users or ultimately generating the whole vector. Some experiments using Large Language Modelss (LLMs) showed promising results in specific contexts [4], but the applicability of general-purpose LLMs for the task has never been investigated.

Contributions. In this paper, we investigate the applicability of LLMs in generating CVSS vectors from CVE descriptions to facilitate and assist analysis. We conduct several experiments in different conditions and with various models. We also explore the feasibility of using text embedding models and traditional Machine Learning (ML) classification models to generate the same vectors, providing a training-based approach. The main contributions of this paper can be summarized as follows.

- We provide the first comprehensive analysis on the capabilities of LLMs in generating CVSS vectors and scores for CVEs.
- We show how Gemma3 is the best-performing model for the task, reaching an accuracy of up to 0.98 in the best-performing CVSS vector elements.
- We discussed the pros and cons of LLM versus an embedding approach. We show how a hybrid approach can strengthen the automatic CVSS generation reaching a mean accuracy of 0.84 through all the CVSS elements.
- We make our code and implementation open-source: `https://github.com/spritz-group/LLM-CVSS`.

Authorized licensed use limited to: Rajamangala Univ of Technology Srivijaya provided by UniNet. Downloaded on October 01,2026 at 17:05:29 UTC from IEEE Xplore. Restrictions apply.

II. Related Works

Since the advent of the first LLMs, researchers have started employing them in a wide range of fields. Different works propose LLM-based solutions for malware detection, program repair, anomaly detection, fuzzing, vulnerability detection, and other cybersecurity-related tasks [5]. Another sector that founds applicability of LLMs is the Cyber-Threat Intelligence (CTI), where generative models can be employed to enhance different processes. LOCALINTEL is a framework that enables CTI organization-specific contextualization [6]. It enables the generation of contextualized information from a repository of global threats (e.g., CVEs) together with a local knowledge graph containing, for instance, configurations of the employed devices. Similarly, Perrina et al. [7] developed AGIR, a tool to automatically generate CTI reports.

Language models have been used to extract intelligence from darknet forums or other unstructured data sources [8]. AttacKG+, a fully automated framework proposed by Zhang et al., allows for the construction of knowledge graphs from CTI using LLMs [9]. Fayyazi et al. proposed a solution to summarize Tactics, Techniques, and Procedures (TTPs) from the MITRE ATT&CK framework boosting LLM with Retrieval-Augmented Generation (RAG) [10].

Some works proposed also the employment of LLMs as security and network experts [11]. Lin et al. designed a solution that allows the retrieval of CVEs and Common Weakness Enumeration (CWE) from a vulnerability description, and use it for vulnerabilities mitigation [12]. Donadel et al. [13] investigated the capabilities of LLMs in understanding and answering questions related to a provided network architecture. However, none of these tools is specifically designed or tested on CVEs classification and ranking.

In cybersecurity, embedding models have proven effective for automating the classification and prioritization of vulnerabilities. By converting vulnerability descriptions into vector representations, these models support tasks such as CVSS scoring and CWE mapping. CVSS-BERT, for example, predicts CVSS metrics using BERT-based classifiers with explainable outputs [14], while V2W-BERT links CVEs to CWEs through transformer-based learning [15]. Such methods highlight the potential of embeddings to streamline vulnerability assessment and support informed security decisions.

III. Background

Common Vulnerabilities and Exposures (CVEs) are publicly disclosed cybersecurity vulnerabilities that have been cataloged to provide a standardized reference for identifying and discussing specific issues. Each CVE is assigned a unique identifier and is maintained by the MITRE Corporation[1] as part of a broader effort to facilitate information sharing across the cybersecurity community.

The Common Vulnerability Scoring System (CVSS) is a standardized framework for rating the severity of security vulnerabilities. It is widely used by organizations and researchers to assess and prioritize the risks associated with specific CVEs. CVSS provides a numerical score that reflects the potential impact and exploitability of a vulnerability.

CVSS scores are computed directly from the CVSS vector, which encodes a set of base metrics describing key characteristics of the vulnerability. These metrics include the Attack Vector (AV), Attack Complexity (AC), Privileges Required (PR), User Interaction (UI), Scope (S), and the impacts on Confidentiality (C), Integrity (I), and Availability (A). An example of a complete vector is the following:

```
CVSS:3.1/AV:N/AC:H/PR:L/UI:R/S:C/C:L/I:L/A:H
```

Each metric is assigned a specific value, and these values are combined using a defined formula to compute the Base Score, which ranges from 0.0 to 10.0. The formula consists of two main subscores: the Exploitability Subscore, derived from AV, AC, PR, and UI; and the Impact Subscore, calculated from C, I, A, and adjusted based on Scope. These subscores are combined using a deterministic set of equations defined in the CVSS specification (currently CVSS v3.1), ensuring that the final score reflects both the ease of exploitation and the potential impact of the vulnerability [16].

IV. Methodology

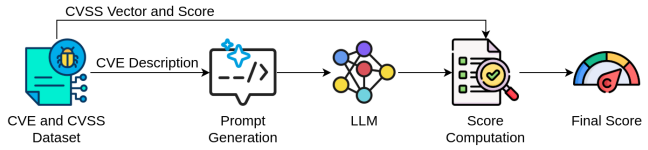
This paper aims to provide an overall evaluation of the capabilities of general-purpose LLMs in classifying CVEs. In particular, we aim to provide an answer to the following research questions.

RQ1 Are general-purpose LLMs capable of classifying CVE description generating the related CVSS vector?

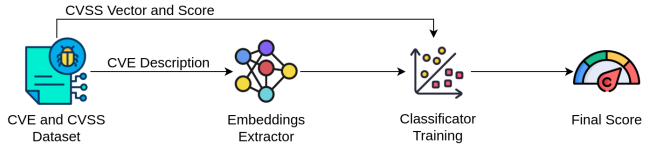
RQ2 Are there any prompt engineering tricks to enhance runtime performance?

RQ3 How "traditional" classification techniques based on textual embeddings perform with respect to LLMs?

We set two scenarios to answer our research queries: *vanilla* and *embeddings*. The former setup directly employs general-purpose LLM without any fine-tuning, while the latter employs different extractors to generate embeddings from CVE descriptions and traditional ML models to classify them.



(a) *Vanilla* architecture.



(b) *Embeddings* architecture.

Fig. 1: Architecture employed in our experiments.

[1]https://cve.mitre.org/

Authorized licensed use limited to: Rajamangala Univ of Technology Srivijaya provided by UniNet. Downloaded on October 01,2026 at 17:05:29 UTC from IEEE Xplore. Restrictions apply.

A. Vanilla LLMs

In the base scenario, we consider only plain and untouched LLMs. A prompt is generated by taking a CVE description as input, which is then fed to the LLM under test, asking to generate the CVSS vector, as shown in Figure 1a. Then, the output is fed to a simple score calculator, which will compute the success rate based on the ground-truth CVSS information.

We performed various experiments in this scenario while changing and improving the employed prompt. The base prompt asked the model to return the complete CVSS vector for one CVE at the time. Then, we tried a few-shot approach to explain the task to the model better. Our prompt is enhanced for this experiment with a list of 3 examples preceding the CVE description under investigation. We selected these examples in such a way that they maximize the variability between the CVSS components while keeping the most commonly known CVEs to ensure sufficient representation in the original training dataset. We also limit to only three examples given the possibly limited capabilities of small LLMs under test, avoiding the risks that can occur with a smaller context window. Moreover, since the CVE description is often short and misses some vulnerability details, we test the effect of enhancing the description by adding CWE descriptions. The hypothesis is that, by knowing the specific CWE exploited by a CVE, it would be possible to evaluate better the risk on specific CVSS vectors. Thus, we extract the CWE identifier from each CVE entry and enrich it with additional information retrieved from the official MITRE database[2]. Specifically, we include the CWE description, common consequences, and potential mitigations. These fields were chosen because they are consistently available across most CWE entries and provide valuable insights into the nature and impact of the vulnerability. We also experiment with a prompt asking the LLM to generate a single vector component. Therefore, eight queries are required to compute the whole CVSS vector in this case.

Finally, we prompt the LLMs with the CVE description and ask them to directly generate the overall CVSS score without classifying each vector component. This approach allows us to assess the models' ability to estimate the risk associated with a given CVE, providing insight into their potential usefulness for vulnerability prioritization.

Regardless of the specific task, each input prompt is constructed by providing information on each CVSS component (in the case of single-component classification, only the relevant one is included). We also include an example of the output format that the LLM should use, and we highlight the need to generate only the relevant information (i.e., vector or component) in the specified format. Since some of the LLMs used also have reasoning capabilities, we filter the response by removing possible `<think>` tags in the response. Furthermore, we consider cases where the model provides an answer in a different format, e.g., by including additional reasonings or specifying the label (e.g., "Network" instead of "N"). We do this by defining a hierarchy of Regex rules, which process and filter the response into the allowed labels for the specific component. All prompts and rules used are shown in our repository.

[2]Example: https://cwe.mitre.org/data/definitions/89.html.

B. Embeddings

Another completely different approach considers the usage of embeddings. Embeddings are numerical vector representations of text that capture their semantic meaning. They allow models to compare, relate, and reason about language by placing similar concepts close together in a high-dimensional space [17]. As shown in Figure 1b, we employed different models to generate embeddings from CVE descriptions. Then, we trained an ML classifier to classify each of the CVSS vector elements, allowing for a reconstruction of the overall vector. It can then be used to compute the CVSS score as per specification [16]. To enhance the capabilities of the models, we also considered the addition of the same CWE information detailed in Section IV-A. Each of these parameters is then embedded and considered as a separate feature we can use for classification and analysis.

C. Experimental Setup

We collected CVEs from the official CVE Github repository[3]. We kept only entries with an associated CVSS v3.1 vector and score, which represents an updated and widely used version of the indicator. Indeed, although CVSS v4.0 exists, not every entry in the dataset contains it. After this preprocessing, we end up with a dataset of 45k entries of CVEs from 2002 to 2025. Figure 2 summarizes the data distribution inside the dataset. In the embedding scenario, we utilized the entire dataset for training, while for the vanilla scenario, which does not involve any training phase, we restricted the test set to the most recent 1000 CVEs. This approach ensures that the CVEs in the test set were not included in the training data of the LLMs, as all data entries date from March 2025 onward.

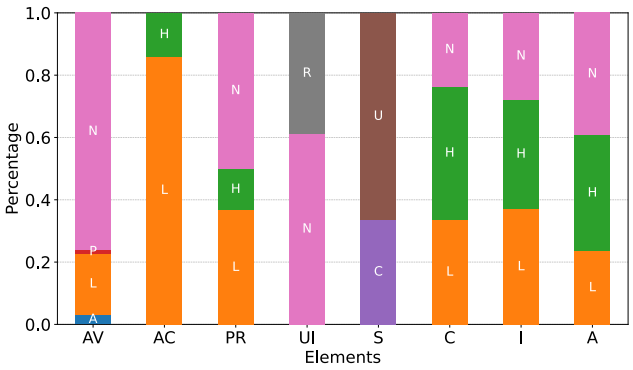


Fig. 2: Distribution of CVSS vector elements in our dataset.[4]

[3]https://github.com/CVEProject/cvelistV5/

[4]Labels: AV (Attack Vector): Network (N), Adjacent (A), Local (L), Physical (P); AC (Attack Complexity): Low (L), High (H); PR (Privileges Required): None (N), Low (L), High (H); UI (User Interaction): None (N), Required (R); S (Scope): Unchanged (U), Changed (C); C/I/A (Confidentiality, Integrity, Availability): None (N), Low (L), High (H).

Authorized licensed use limited to: Rajamangala Univ of Technology Srivijaya provided by UniNet. Downloaded on October 01,2026 at 17:05:29 UTC from IEEE Xplore. Restrictions apply.

In the vanilla scenario, we employed different LLMs, both proprietary and open-weights. In particular, we considered DeepSeek R1 (8B parameters) [18], Gemma 3 (12B parameters) [19], GPT 4o Mini [20], Llama 3.2 (3B parameters) [21], Mistral [22], Phi 4 [23], and Qwen 2.5 (14B parameters) [24].

In the embeddings scenarios, instead, we employed three different models to extract the embeddings: all-miniLM (L6, version 2), nomic, and ATTACK-BERT.[5] For classification, we tested different models (XGBoost, Random Forest, Logistic Regression, and some simple Neural Networks) and ended up using XGBoost [25], which shows better results. We act a split of 80%/20% between training and testing data. We employed accuracy as the metric to show our results, representing the ratio between the number of correctly classified samples and the total number of samples for a specific vector element.

V. Results

In this section, we show the results of the experiments in both the vanilla and embedding case. Moreover, we propose a comparison of the best approaches in two scenarios.

A. Vanilla LLMs

We start by analyzing the most basic scenario possible, i.e., asking an LLM to fill each of the CVSS vector elements together. Results of the vanilla scenarios are shown in Table I. Generally, we can see how all the considered LLMs performed well, despite some important differences between vector elements. In particular, while the AV is detected correctly by almost all the considered models, PR is more tricky. Regarding the impact metrics (C, I, A), the results are not satisfying. While C and I exhibit reasonable accuracy when using the Gemma 3 model, A achieve only 0.47 in the best case. However, Gemma 3 is overall the best-performing model.

TABLE I: Accuracies in the vanilla scenario for different LLMs in generating the CVSS vector in one single shot.

Model	AV	AC	PR	UI	S	C	I	A
DeepSeek	0.91	0.86	0.46	0.63	0.55	0.35	0.29	0.33
Gemma 3	0.95	**0.92**	**0.48**	**0.95**	**0.82**	0.62	**0.70**	**0.47**
GPT 4o Mini	0.97	**0.92**	**0.48**	0.94	0.65	0.35	0.31	0.34
Llama 3.2	0.53	0.52	0.41	0.66	0.57	0.23	0.22	0.24
Mistral	**0.98**	**0.92**	0.47	0.71	0.41	0.62	0.61	0.33
Phi 4	0.95	**0.92**	**0.48**	0.76	0.45	0.24	0.23	0.29
Qwen 2.5	0.97	0.91	0.47	0.76	0.61	**0.63**	0.22	0.26

Moreover, we tested some prompt engineering enhancements to see if the LLMs might benefit from more information in the prompt or for a more precise question. Results are shown in Figure 3. In the case of AV, all the prompts show similar results, with a negligible advancement of the prompt containing CWEs. Similarly, AC and PR show steady accuracies for CWE and few shots compared to the base prompt, while asking one element at a time significantly decreases the accuracies. In the others, instead, CWE performed poorly with a performance reduction particularly marked in the impact metrics. The few shots, instead, reduce some of the accuracies without creating a sensitive increase in the others. Moreover, asking for one component at a time does not add any benefits since it shows low results at the cost of an increased amount of queries. These experiments suggest that LLMs already has all the needed information to compute as best as possible the CVSS vector, and additional data on the prompt are not recommended.

[5]A version of the widely used BERT model fine-tuned on cybersecurity tasks: https://huggingface.co/basel/ATTACK-BERT

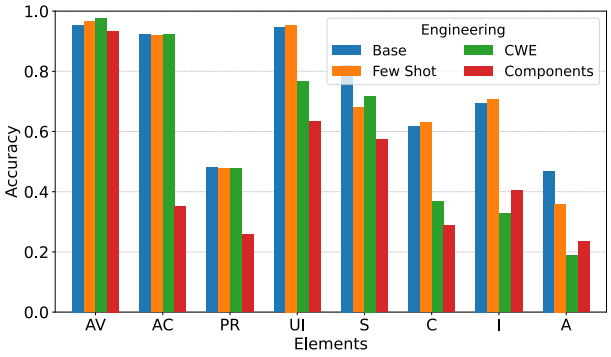


Fig. 3: Accuracy with different prompt engineering techniques using the best performing model (Gemma 3).

B. Embeddings

Similarly, results for the embeddings architecture are shown in Table II. The employed model for classification is XGBoost [25]. First of all, we can see how results do not change much when changing the embedding model, even though we can see how nomic is performing slightly worse than the other two which returned almost the same average accuracy with some tiny variations in the components. The best model, all-MiniLM, obtained the best accuracy of 0.89 in the AV element, while PR is the worst element with an accuracy of 0.73.

With respect to the enhancement solutions applied on the prompt in the Vanilla scenario which reduced accuracy, when using embeddings adding CWE has only a slight effect on the final accuracy, as shown in Figure 4. Due to this negligible improvement, we chose as default the base version which reduced the computation required.

TABLE II: Accuracies in the embeddings scenario for different embeddings models in classifying each vector element alone.

Embedding	AV	AC	PR	UI	S	C	I	A
nomic	0.87	**0.87**	0.68	0.87	0.85	**0.78**	**0.78**	**0.77**
all-MiniLM	**0.89**	**0.87**	**0.73**	0.87	**0.86**	0.77	0.77	**0.77**
attack-bert	**0.89**	**0.87**	0.70	**0.88**	**0.86**	0.77	0.76	0.76

C. Comparison

As depicted in Section IV, different enhancements could strengthen the results and have been applied to our scenarios. Results are shown in Figure 5. We show the results for the best LLM of the vanilla scenario (i.e., Gemma 3) and the best embedding model (i.e., all-MiniLM).

Authorized licensed use limited to: Rajamangala Univ of Technology Srivijaya provided by UniNet. Downloaded on October 01,2026 at 17:05:29 UTC from IEEE Xplore. Restrictions apply.

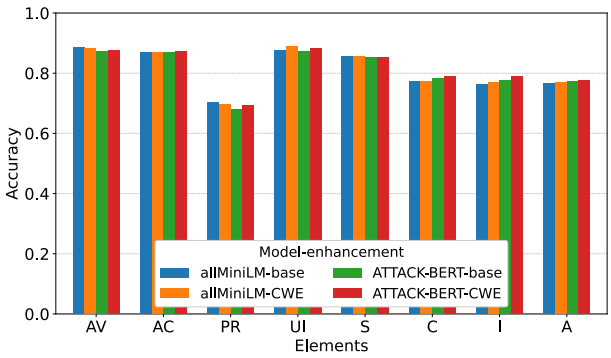


Fig. 4: Accuracy provided by base embeddings versus embeddings enhanced with CWE.

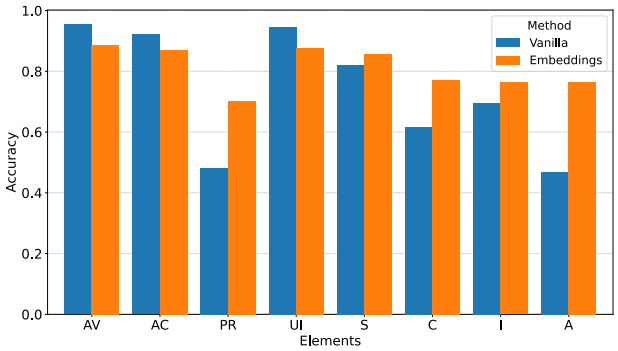


Fig. 5: Comparison between the best vanilla (Gemma 3) and the best embedding model (all-MiniLM + XGBoost).

While results for singular vector elements depend on the element itself, in the vanilla case we can see a higher variation with respect to the embedding scenario. While AV shows lower results than LLMs, other elements where language models struggle to provide a good classification show instead reasonable results in this scenario. An example is, for instance, the PR element, where accuracy increases from 0.48 to 0.73. Another interesting result relates to the accuracy for the A impact, with an increase from 0.47 to 0.77. In other cases, such as AV, AC, and UI, the accuracy is higher when using LLMs. Table III shows the mean accuracy over all the vector components for the already presented solutions, in addition to a hybrid approach. In this solution, we generate AV, AC, and UI from LLMs, while PR, S, and the impact metrics are generated through embeddings. We observe an improvement in the final accuracy with this method, indicating that a hybrid strategy yields the best results.

TABLE III: Accuracy in the different proposed approaches.

Approach	Mean Accuracy
Vanilla	0.738
Embeddings	0.811
Hybrid	**0.835**

D. Direct Scoring

As an alternative to component-wise classification, we also experimented with direct CVSS scoring by prompting language models to estimate the overall score of a vulnerability. While this method provides less interpretability compared to decomposing the vector, it offers insights into how well LLMs understand and assess vulnerability criticality. This task serves as a proxy to evaluate their ability to rank CVEs for prioritization. Table IV reports the Mean Squared Error (MSE) and Mean Absolute Error (MAE) for each model. Results show that all models deviate from the ground truth, but Qwen 2.5 and GPT 4o yield more accurate scores, indicating stronger potential for scoring tasks.

TABLE IV: Direct CVSS scoring performance of LLMs.

Model	MSE	MAE
DeepSeek	32.84	3.51
Gemma 3	2.65	1.24
GPT 4o Mini	2.47	1.21
Llama 3.2	3.07	1.33
Mistral	3.13	1.32
Phi 4	2.62	1.21
Qwen 2.5	**2.32**	**1.06**

VI. Discussion

The results presented in Section V need some considerations. First of all, they suggest that a partial affirmative answer to **RQ1**, since LLMs can be safely employed to generate vector elements such as AV, AC, and UI with a good confidence thanks to accuracies surpassing 0.90. On the contrary, LLMs struggle with other elements such as PR and the A impact, where accuracies below 0.50 do not offer a reliable classification.

We tried different enhancements on the prompt to answer **RQ2**. As shown in Figure 3, results were generally not improving with more complex prompts, and often getting worse. This suggested that a base prompt is the best choice to provide high accuracies and to limit the queries needed to generate each vector. Similarly, even in the embedding scenario adding CWE information is not needed to achieve good accuracies.

The comparison in Figure 5 allows answering **RQ3**. However, the answer depends on the considered vector element. For AV, AC, and UI there is a significant increase in the results using LLMs. However, the other elements do not show the same trend, and the embedding approach results in better results, suggesting that a hybrid approach could be the best solution, as shown in Table III.

Although one might expect LLMs to excel at interpreting subjective aspects due to their ability to reason through context and nuance, this doesn't always translate into accurate classification. Metrics like "Low" vs. "High" confidentiality (C) or integrity (I) impacts are inherently vague and domain-dependent. Even official CVSS v3.1 scores can vary between analysts, highlighting the subjective nature of the task [1].

Authorized licensed use limited to: Rajamangala Univ of Technology Srivijaya provided by UniNet. Downloaded on October 01,2026 at 17:05:29 UTC from IEEE Xplore. Restrictions apply.

Moreover, LLMs may also overfit to vague interpretations or rhetorical cues not closely tied to CVSS definitions. In contrast, embedding-based models, particularly when fine-tuned on security data, tend to capture more subtle patterns linked to severity, suggesting improved alignment with statistical regularities. This subjectivity extends to several CVSS vector elements. While some, like AV, are more objective, others (such as PR) depend heavily on analyst judgment. Different researchers may weigh factors differently, leading to inconsistent classifications.

Additionally, human analysts often struggle with accurate CVSS scoring [1], resulting in mislabeled CVEs. In such cases, LLMs or embedding-based models may even outperform the "ground truth", suggesting that generative models could eventually exceed human performance—a possibility worth exploring further.

VII. Conclusions

This study explored the potential of LLMs in automating the generation of CVSS vectors from CVE descriptions, a task that is increasingly vital due to the growing volume of reported vulnerabilities. Our experiments demonstrated that while LLMs can achieve high accuracy on certain CVSS elements, particularly those with more objective criteria, they struggle with more subjective dimensions compared to traditional embedding-based methods. Notably, a hybrid approach that combines LLMs with embedding models proved to be a promising direction, offering improved overall performance. These findings underscore the challenges of CVSS scoring and suggest refining hybrid models with domain tuning and analyst feedback to improve real-world reliability. Moreover, in this research, we only considered the most widely used elements of the CVSS metric. However, temporal and environmental metrics are also available [16], even if not mandatory and rarely employed. Especially for this reason, LLMs can represent a good solution to fill their values without augmenting the burden on the user and will be investigated in further studies.

References

[1] J. Wunder, A. Kurtz, C. Eichenmüller, F. Gassmann, and Z. Benenson, "Shedding light on cvss scoring inconsistencies: A user-centric study on evaluating widespread security vulnerabilities," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 1102–1121.

[2] Kaaviya, "Over 40,000 cves published in 2024, marking a 38% increase from 2023," *Cyber Press*, 2025. [Online]. Available: https://cyberpress.org/over-40000-cves-published-in-2024/

[3] Fortress Information Security, "Nvd analysis report," 2024, accessed: 2025-04-08. [Online]. Available: https://www.fortressinfosec.com/nvd-analysis-report

[4] R. Ghosh, H.-M. von Stockhausen, M. Schmitt, G. M. Vasile, S. K. Karn, and O. Farri, "Cve-llm: Ontology-assisted automatic vulnerability evaluation using large language models," *arXiv preprint arXiv:2502.15932*, 2025.

[5] J. Zhang, H. Bu, H. Wen, Y. Liu, H. Fei, R. Xi, L. Li, Y. Yang, H. Zhu, and D. Meng, "When llms meet cybersecurity: A systematic literature review," *Cybersecurity*, vol. 8, no. 1, pp. 1–41, 2025.

[6] S. Mitra, S. Neupane, T. Chakraborty, S. Mittal, A. Piplai, M. Gaur, and S. Rahimi, "Localintel: Generating organizational threat intelligence from global and local cyber knowledge," *arXiv preprint arXiv:2401.10036*, 2024.

[7] F. Perrina, F. Marchiori, M. Conti, and N. V. Verde, "Agir: Automating cyber threat intelligence reporting with natural language generation," in *2023 IEEE International Conference on Big Data (BigData)*. IEEE, 2023, pp. 3053–3062.

[8] G. Siracusano, D. Sanvito, R. Gonzalez, M. Srinivasan, S. Kamatchi, W. Takahashi, M. Kawakita, T. Kakumaru, and R. Bifulco, "Time for action: Automated analysis of cyber threat intelligence in the wild," *arXiv preprint arXiv:2307.10214*, 2023.

[9] Y. Zhang, T. Du, Y. Ma, X. Wang, Y. Xie, G. Yang, Y. Lu, and E.-C. Chang, "Attackg+: Boosting attack knowledge graph construction with large language models," *arXiv preprint arXiv:2405.04753*, 2024.

[10] R. Fayyazi, R. Taghdimi, and S. J. Yang, "Advancing ttp analysis: harnessing the power of large language models with retrieval augmented generation," in *2024 Annual Computer Security Applications Conference Workshops (ACSAC Workshops)*. IEEE, 2024, pp. 255–261.

[11] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *2024 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2024, pp. 862–880.

[12] Y.-Z. Lin, M. Mamun, M. A. Chowdhury, S. Cai, M. Zhu, B. S. Latibari, K. I. Gubbi, N. N. Bavarsad, A. Caputo, A. Sasan *et al.*, "Hw-v2w-map: Hardware vulnerability to weakness mapping framework for root cause analysis with gpt-assisted mitigation suggestion," *arXiv preprint arXiv:2312.13530*, 2023.

[13] D. Donadel, F. Marchiori, L. Pajola, and M. Conti, "Can llms understand computer networks? towards a virtual system administrator," in *2024 IEEE 49th Conference on Local Computer Networks (LCN)*. IEEE, 2024, pp. 1–10.

[14] M. R. Shahid and H. Debar, "Cvss-bert: Explainable natural language processing to determine the severity of a computer security vulnerability from its description," in *2021 20th IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 2021, pp. 1600–1607.

[15] S. S. Das, E. Serra, M. Halappanavar, A. Pothen, and E. Al-Shaer, "V2w-bert: A framework for effective hierarchical multiclass classification of software vulnerabilities," in *2021 IEEE 8th International Conference on Data Science and Advanced Analytics (DSAA)*. IEEE, 2021, pp. 1–12.

[16] C. Group *et al.*, "Common vulnerability scoring system version 3.1: Specification document," in *FIRST-Forum of Incident Response and Security Teams, Cary, USA, Standard*, 2019.

[17] M. Kusner, Y. Sun, N. Kolkin, and K. Weinberger, "From word embeddings to document distances," in *International conference on machine learning*. PMLR, 2015, pp. 957–966.

[18] D. Guo, D. Yang, H. Zhang, J. Song, R. Zhang, R. Xu, Q. Zhu, S. Ma, P. Wang, X. Bi *et al.*, "Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning," *arXiv preprint arXiv:2501.12948*, 2025.

[19] G. Team, T. Mesnard, C. Hardin, R. Dadashi, S. Bhupatiraju, S. Pathak, L. Sifre, M. Rivière, M. S. Kale, J. Love *et al.*, "Gemma: Open models based on gemini research and technology," *arXiv preprint arXiv:2403.08295*, 2024.

[20] A. Hurst, A. Lerer, A. P. Goucher, A. Perelman, A. Ramesh, A. Clark, A. Ostrow, A. Welihinda, A. Hayes, A. Radford *et al.*, "Gpt-4o system card," *arXiv preprint arXiv:2410.21276*, 2024.

[21] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar *et al.*, "Llama: Open and efficient foundation language models," *arXiv preprint arXiv:2302.13971*, 2023.

[22] A. Q. Jiang, A. Sablayrolles, A. Roux, A. Mensch, B. Savary, C. Bamford, D. S. Chaplot, D. d. l. Casas, E. B. Hanna, F. Bressand *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[23] M. Abdin, J. Aneja, H. Behl, S. Bubeck, R. Eldan, S. Gunasekar, M. Harrison, R. J. Hewett, M. Javaheripi, P. Kauffmann *et al.*, "Phi-4 technical report," *arXiv preprint arXiv:2412.08905*, 2024.

[24] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei *et al.*, "Qwen2. 5 technical report," *arXiv preprint arXiv:2412.15115*, 2024.

[25] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," in *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, 2016, pp. 785–794.

Authorized licensed use limited to: Rajamangala Univ of Technology Srivijaya provided by UniNet. Downloaded on October 01,2026 at 17:05:29 UTC from IEEE Xplore. Restrictions apply.